기타 다항식 커밋먼트 스킴들 개요

소개

이 문서는 KZG10과 Marlin PC 외에 arkworks-rs/poly-commit 라이브러리에서 구현된 다른 다항식 커밋먼트 스킴들의 특징과 사용 사례를 다룹니다. 각 스킴은 서로 다른 암호학적 가정과 성능 특성을 가지며, 특정 상황에서 최적의 선택이 될 수 있습니다.

지원되는 스킴들 비교

전체 스킴 비교표

| 스킴 | 기반 가정 | 증명 크기 | 검증 시간 | Setup | 특징 |

|------|----------|----------|----------|------|------|

| **KZG10** | Pairing + Trusted Setup | O(1) | O(1) | Trusted | 가장 효율적, 신뢰할 수 있는 setup 필요 |

| **Marlin PC** | Pairing + Trusted Setup | O(1) | O(1) | Trusted | KZG10 + degree bound 강제 |

| **Sonic PC** | Pairing + Trusted Setup | O(1) | O(1) | Trusted | KZG10 + 배치 최적화 + degree bound |

| **IPA PC** | Discrete Log | O(log d) | O(log d) | Transparent | Trusted setup 불필요, 더 큰 증명 |

| **Hyrax** | Discrete Log | O(√d) | O(√d) | Transparent | Multilinear 전용, 정사각 행렬 제약 |

| **Multilinear PC** | Pairing | O(log d) | O(log d) | Transparent | Multilinear 전용, pairing 기반 |

| **Ligero** | Hash Functions | O(√d) | O(√d) | Transparent | Post-quantum 안전, 큰 증명 |

| **Brakedown** | Hash Functions | O(√d) | O(√d) | Transparent | Ligero 개선, 더 나은 concrete 효율성 |

Sonic PC

특징

Sonic PC는 KZG10을 기반으로 하되, 배치 검증과 degree bound 강제 기능을 추가한 스킴입니다.

```
/// Sonic PC의 핵심 구조
pub struct SonicKZG10<E: Pairing, P: DenseUVPolynomial<E::ScalarField>> {
_engine: PhantomData<E>,
_poly: PhantomData<P>,
}
```

주요 개선사항

- **배치 검증**: 여러 증명을 동시에 효율적으로 검증
- **Degree bound 강제**: Marlin과 유사한 방식으로 차수 제한 적용
- **Negative G1 powers 회피**: AuroraLight 논문의 최적화 적용

사용 사례

- 여러 다항식을 동시에 검증해야 하는 경우
- Degree bound가 중요한 zk-SNARK 시스템
- KZG10의 효율성을 유지하면서 추가 기능이 필요한 경우

IPA PC (Inner Product Argument)

특징

IPA PC는 이산 로그 문제의 어려움에 기반한 투명한 다항식 커밋먼트 스킴입니다.

```
/// IPA PC의 핵심 구조
pub struct InnerProductArgPC<G: AffineRepr, D: Digest, P: DenseUVPolynomial<G::S
_projective: PhantomData<G>,
_digest: PhantomData<D>,
_poly: PhantomData<P>,
}
```

장점

- **투명한 setup**: 신뢰할 수 있는 setup이 불필요
- **범용성**: 임의의 타원곡선 그룹에서 동작
- **보안성**: 이산 로그 가정만 필요

단점

- **로그 크기**: 증명 크기와 검증 시간이 O(log d)
- **성능**: KZG10 대비 느린 성능

사용 사례

- 신뢰할 수 있는 setup을 피하고 싶은 경우
- 투명성이 중요한 시스템
- Pairing이 지원되지 않는 환경

Multilinear Polynomial Commitment

Hyrax vs Multilinear PC 비교

Hyrax

```
/// Hyrax의 핵심 구조
pub struct HyraxPC<G: AffineRepr, P: MultilinearExtension<G::ScalarField>> {
_phantom: PhantomData<(G, P)>,
}
```

특징:

- **이산 로그 기반**: Pairing 불필요
- **정사각 행렬 제약**: 변수 개수가 짝수여야 함
- **√d 복잡도**: 증명 크기와 검증 시간이 O(√d)
- **투명한 setup**: 신뢰할 수 있는 setup 불필요

Multilinear PC

```
/// Multilinear PC의 핵심 구조
pub struct MultilinearPC<E: Pairing> {
_engine: PhantomData<E>,
}
```

특징:

- **Pairing 기반**: 더 효율적인 검증
- **로그 복잡도**: 증명 크기와 검증 시간이 O(log d)
- **유연성**: 변수 개수 제약 없음

- **투명한 setup**: 신뢰할 수 있는 setup 불필요

사용법 차이점

Hyrax 사용 예제

```
// Hyrax는 짝수 개의 변수만 지원
let num_vars = 4; // 반드시 짝수
let polynomial = DenseMultilinearExtension::rand(num_vars, &mut rng);


// Setup (투명)
let pp = HyraxPC::setup(max_degree, Some(num_vars), &mut rng)?;
let (ck, vk) = HyraxPC::trim(&pp, supported_degree, 0, None)?;


// Commit (randomness 필수)
let (commitments, states) = HyraxPC::commit(&ck, &[labeled_poly], Some(&mut rng)
```

Multilinear PC 사용 예제

```
// Multilinear PC는 임의 개수의 변수 지원
let num_vars = 5; // 홀수도 가능
let polynomial = DenseMultilinearExtension::rand(num_vars, &mut rng);


// Setup (투명)
let pp = MultilinearPC::setup(num_vars, &mut rng);
```

```
let (ck, vk) = MultilinearPC::trim(&pp, num_vars);


// Commit

let commitment = MultilinearPC::commit(&ck, &polynomial);
```

Linear Code 기반 스킴들

Ligero

특징

- **Reed-Solomon 코드 기반**: 선형 코드의 성질 활용
- **Post-quantum 안전**: 해시 함수만 사용하여 양자 내성
- **투명한 setup**: 신뢰할 수 있는 setup 불필요

구현 방식

```
/// Ligero 파라미터

pub struct LigeroPCParams<F, C, H> {

sec_param: usize, // 보안 파라미터

rho_inv: usize, // 코드 레이트의 역수

check_well_formedness: bool, // Well-formedness 체크 여부

// ... 해시 파라미터들

}
```

핵심 아이디어:

1. 다항식 계수를 행렬로 배치
2. 각 행에 Reed-Solomon 인코딩 적용
3. 인코딩된 열들을 Merkle tree로 커밋
4. 랜덤 선형 결합으로 검증

성능 특성

- **증명 크기**: O(√d)
- **검증 시간**: O(√d)
- **통신 복잡도**: 상대적으로 큼
- **계산 복잡도**: FFT 기반으로 효율적

Brakedown

특징

- **Ligero의 개선**: 더 나은 concrete 효율성
- **재귀적 구조**: 다단계 인코딩으로 성능 향상
- **유연한 파라미터**: 다양한 보안-효율성 트레이드오프

구현 방식

```
/// Brakedown 파라미터
pub struct BrakedownPCParams<F, C, H> {
alpha: (usize, usize), // 첫 번째 확장 비율
beta: (usize, usize), // 두 번째 확장 비율
rho_inv: (usize, usize), // 전체 코드 레이트
base_len: usize, // 재귀 종료 조건
```

```
// ... 희소 행렬들과 해시 파라미터들

}
```

핵심 개선사항:

1. **재귀적 인코딩**: 여러 단계로 나누어 효율성 향상
2. **희소 행렬 사용**: 메모리와 계산 효율성 개선
3. **최적화된 파라미터**: 실제 사용에 적합한 구체적 파라미터

Ligero vs Brakedown 비교

측면	Ligero	Brakedown
인코딩 방식	단일 단계 Reed-Solomon	재귀적 다단계 인코딩
행렬 구조	밀집 행렬	희소 행렬
메모리 사용량	높음	낮음
Setup 복잡도	단순	복잡
Concrete 효율성	보통	우수
파라미터 유연성	제한적	높음

사용 예제

Ligero 사용법

```rust
use ark_poly_commit::linear_codes::{UnivariateLigero, LinearCodePCS};

// Setup
```

```
let max_degree = 1023;

let pp = LinearCodePCS::<UnivariateLigero<F, C, H>, F, P, C, H>::setup(

max_degree, None, &mut rng

)?;



// 또는 직접 파라미터 설정

let ligero_params = LigeroPCParams::new(

128, // 보안 파라미터

4, // rho_inv (코드 레이트 = 1/4)

true, // well-formedness 체크

leaf_hash_param,

two_to_one_hash_param,

col_hash_params,

);
```

Brakedown 사용법

```
use ark_poly_commit::linear_codes::{MultilinearBrakedown, LinearCodePCS};



// 기본 파라미터로 setup

let brakedown_params = BrakedownPCParams::default(

&mut rng,

poly_len,

true, // well-formedness 체크
```

```
leaf_hash_param,
two_to_one_hash_param,
col_hash_params,
);


// 또는 커스텀 파라미터
let brakedown_params = BrakedownPCParams::new(
128, // 보안 파라미터
(178, 1000), // alpha
(61, 1000), // beta
(1521, 1000), // rho_inv
30, // base_len
// ... 기타 파라미터들
);
```

스킴 선택 가이드

성능 우선 (Trusted Setup 허용)

1. **KZG10**: 가장 기본적이고 효율적
2. **Marlin PC**: Degree bound가 필요한 경우
3. **Sonic PC**: 배치 검증이 중요한 경우

투명성 우선 (Trusted Setup 회피)

1. **IPA PC**: 범용적이고 안정적
2. **Multilinear PC**: Multilinear 다항식 전용
3. **Hyrax**: 더 큰 증명을 감수할 수 있는 경우

Post-Quantum 안전성 필요

1. **Brakedown**: 최신 기술, 더 나은 효율성
2. **Ligero**: 검증된 기술, 단순한 구조

특수 용도

- **Multilinear 다항식**: Hyrax 또는 Multilinear PC
- **매우 큰 다항식**: Linear code 기반 스킴들
- **제한된 계산 환경**: IPA PC
- **양자 내성 필요**: Ligero 또는 Brakedown

결론

각 다항식 커밋먼트 스킴은 고유한 장단점을 가지며, 특정 사용 사례에 최적화되어 있습니다. 선택 시 고려해야 할 주요 요소는:

1. **보안 가정**: Pairing, 이산 로그, 해시 함수
2. **Setup 요구사항**: Trusted vs Transparent
3. **성능 요구사항**: 증명 크기, 검증 시간
4. **다항식 타입**: Univariate vs Multilinear
5. **미래 호환성**: Post-quantum 안전성

이러한 요소들을 종합적으로 고려하여 프로젝트의 요구사항에 가장 적합한 스킴을 선택하는 것이 중요합니다.

관련 문서

기본 이해

- **프로젝트 개요**: 모든 스킴들의 기본 특징과 비교표

주요 스킴 분석

- **KZG10 구현**: 가장 기본적인 페어링 기반 스킴의 상세 분석
- **Marlin PC 구현**: KZG10의 고급 버전인 Marlin PC 분석

실습 및 참고

- **사용 예제**: 각 스킴별 실제 사용법과 성능 비교
- **암호학적 배경**: 각 스킴의 보안 가정과 수학적 기반
- **용어집**: 모든 스킴의 용어와 표기법 통합 정리